

vLLM: LLM Production Serving



What is vLLM

open-source



The **High-Throughput** and **Memory-Efficient** inference and serving engine for LLMs

Easy, fast, and cost-efficient LLM serving for everyone.

[Get Started](#) →

[Documentation](#)



Easy

Deploy the widest range of open-source models on any hardware.

Includes a drop-in OpenAI-compatible API for instant integration.



Fast

Maximize throughput with PagedAttention. Advanced scheduling and continuous batching ensure peak GPU utilization.



Cost Efficient

Slash inference costs by maximizing hardware efficiency. We make high-performance LLMs affordable and accessible to everyone.

Simple to Serve with vLLM: Installation

vllm.ai

Build

Stable (0.29.0)

Nightly

Platform

CUDA

ROCm

XPU

CPU

Package

Python (uv)

Python

Docker

CUDA Version

CUDA 13.0

CUDA 12.9

Run this Command:



```
uv pip install vllm --torch-backend auto
```

Simple to Serve with vLLM: Setup serving

```
vllm serve Qwen/Qwen3.6-35B-A3B \  
  --tensor-parallel-size 8 \  
  --max-model-len 262144 \  
  --reasoning-parser qwen3
```

- context lengths of up to 262,144 tokens
- output parser optimized for reasoning models (specifically tailored for qwen3 formatting)

```
vllm serve Qwen/Qwen3.5-397B-A17B-FP8 \  
  -dp 8 \  
  --enable-expert-parallel \  
  --mm-encoder-tp-mode data \  
  --mm-processor-cache-type shm \  
  --reasoning-parser qwen3 \  
  --enable-prefix-caching
```

- dp: Data Parallelism
 - Expert Parallelism (EP) for Mixture of Experts (MoE)
 - Tensor Parallelism (TP) mode for the Multi-Modal (MM) encoder
 - Multi-Modal processor's cache type to Shared Memory (shm).
-
- Turns on automatic prefix caching in the KV (Key-Value) cache

OpenAI API Compatible Server

```
import time
from openai import OpenAI

client = OpenAI(
    api_key="EMPTY",
    base_url="http://localhost:8000/v1",
    timeout=3600
)

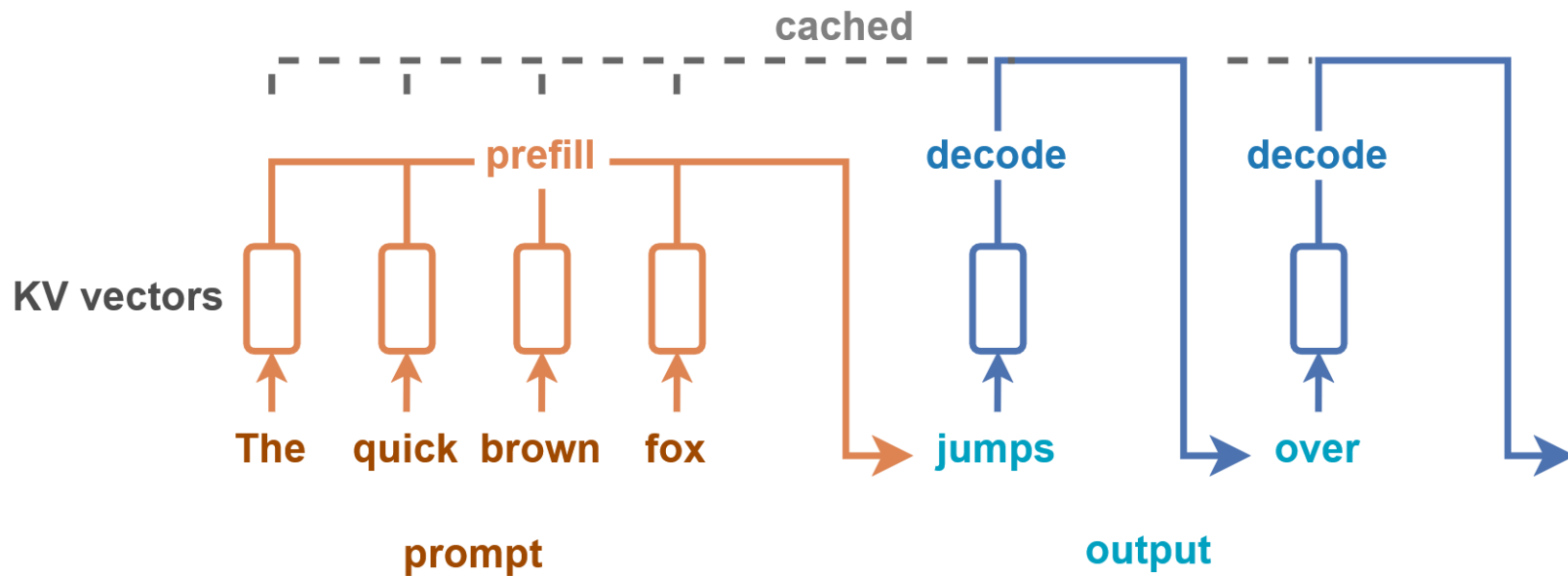
messages = [
    {
        "role": "user",
        "content": [
            {
                "type": "image_url",
                "image_url": {
                    "url": "https://ofasys-multimodal-wlcb-3-toshanghai.oss-accele"
                }
            },
            {
                "type": "text",
                "text": "Read all the text in the image."
            }
        ]
    }
]

start = time.time()
response = client.chat.completions.create(
    model="Qwen/Qwen3.5-397B-A17B",
    messages=messages,
    max_tokens=2048
)
print(f"Response costs: {time.time() - start:.2f}s")
print(f"Generated text: {response.choices[0].message.content}")
```



LLM Serving Challenges

Challenge 1: Prefill vs. Decode asymmetry



Challenge 1: Prefill vs. Decode asymmetry

- LLM generation happens in two phases in different resource profiles:
Prefill phase that processes the prompt to generate first token
Decode phase that generates tokens sequentially.

PHASE	BOTTLENECK	SCALES WITH	METRIC DOMINATED
Prefill	Compute (FLOPs)	Prompt length ²	TTFT (Time to First Token)
Decode	Memory Bandwidth	Model size × steps	ITL (Inter-Token Latency)

Challenge 2: KV cache memory waste

- Conventional systems pre-allocate a contiguous chunk of memory for the maximum possible sequence length.
- This causes massive waste through **reservation**, **internal fragmentation**, and **external fragmentation**.

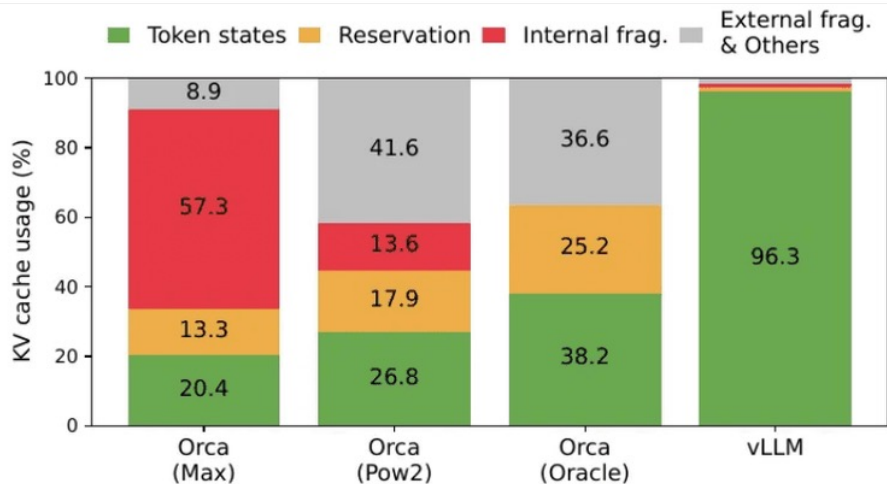


Figure 2. Average percentage of memory wastes in different LLM serving systems during the experiment in §6.2.

Orca is a distributed serving system for large language models

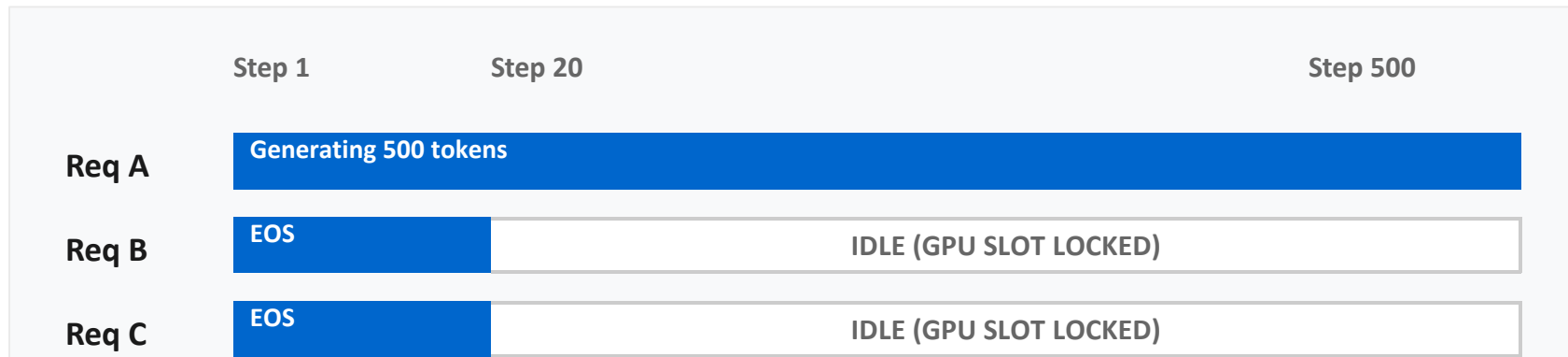
internal frag: wasted bytes are *inside* a valid allocation

external frag: free KV-cache memory that exists, but is split into small non-contiguous gaps

reservation: reserved for growing without needing a new allocation

Challenge 3: Batching with varying length of outputs

- Because decode steps generate one token at a time, batching requests together amortizes the massive memory-bandwidth cost of loading model weights.
- Static batching waits for the longest sequence to finish before starting a new batch, wasting GPU cycles on early-finishing requests.

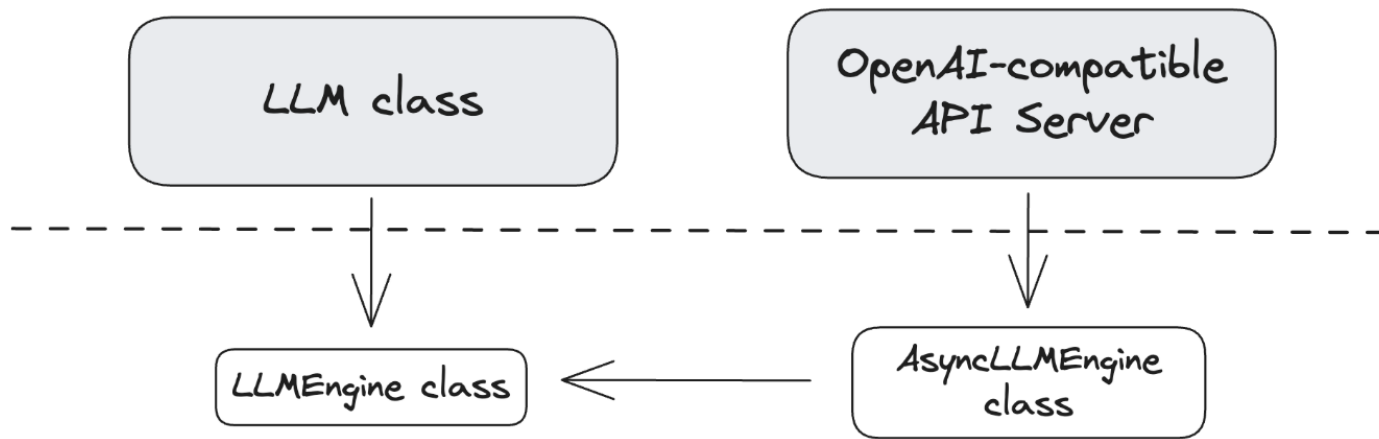




vLLM Architecture and Design

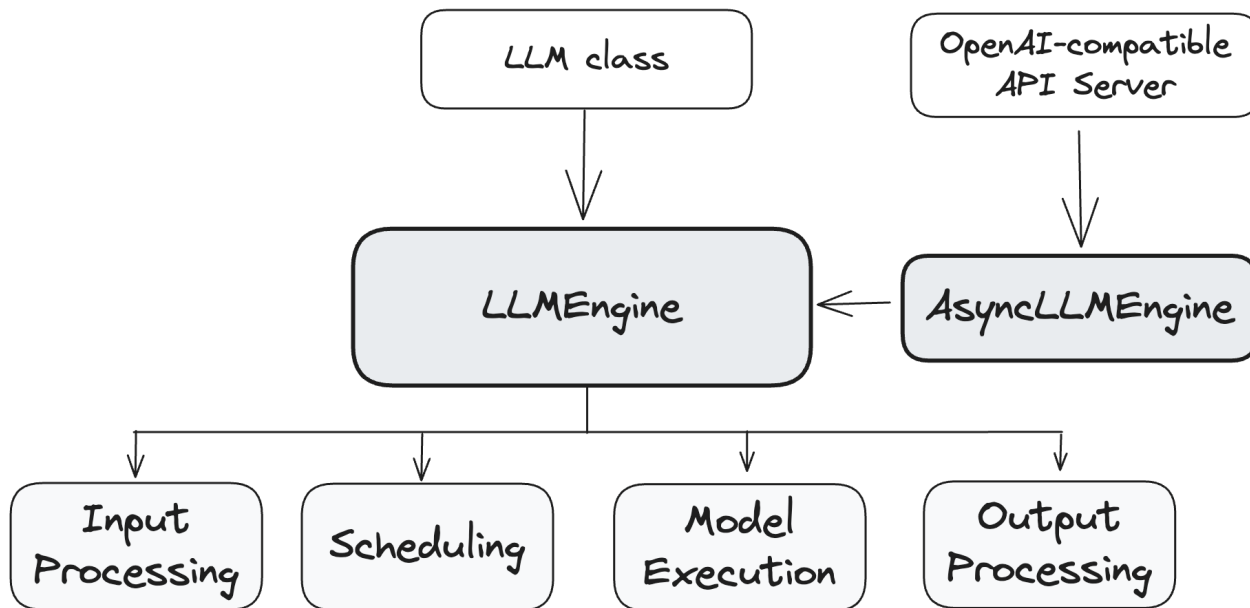
vLLM architecture overview

Entrypoints

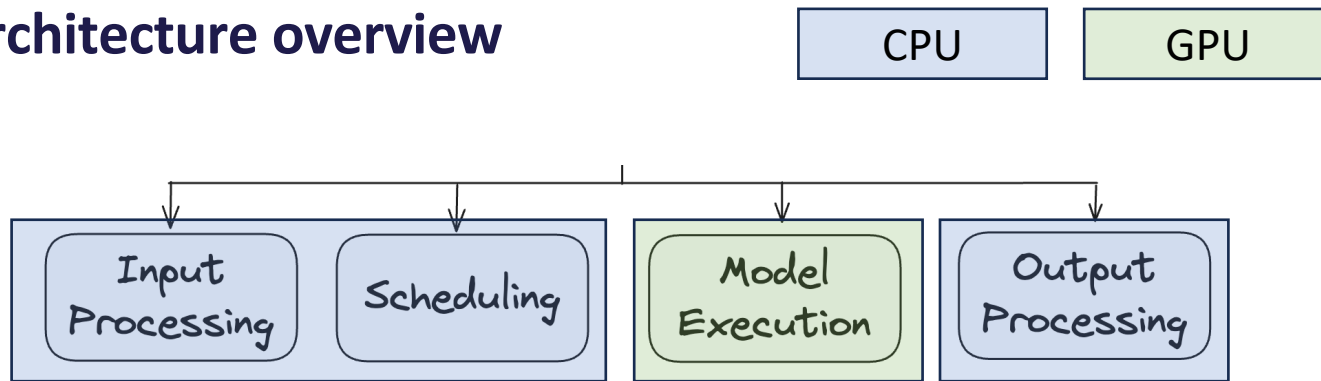


vLLM architecture overview

LLM Engine

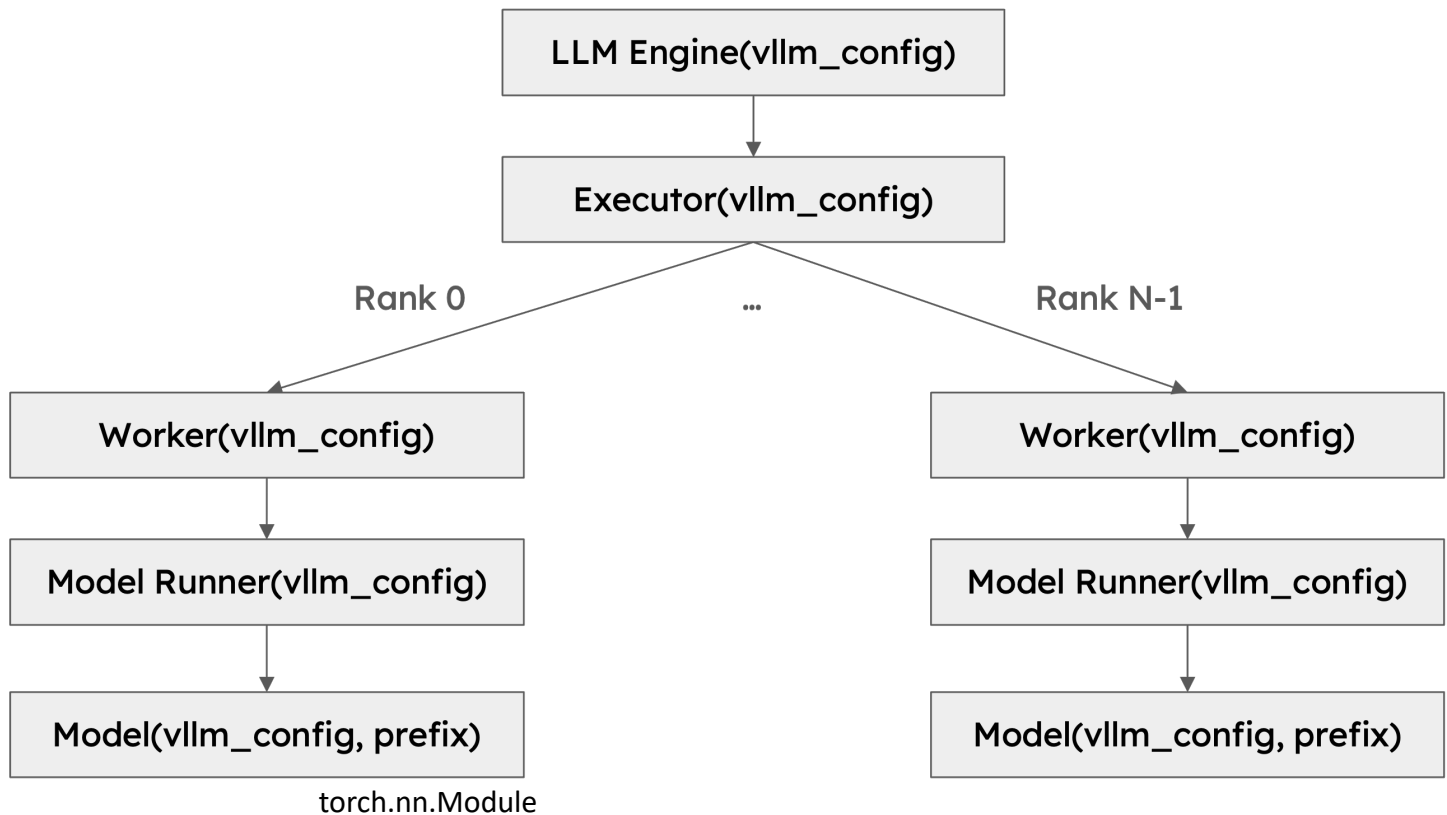


vLLM architecture overview



- **Input Processing:** Handles tokenization of input text using the specified tokenizer.
- **Scheduling:** Chooses which requests are processed in each step.
- **Model Execution:** Manages the execution of the language model, including distributed execution across multiple GPUs.
- **Output Processing:** Processes the outputs generated by the model, decoding the token IDs from a language model into human-readable text.

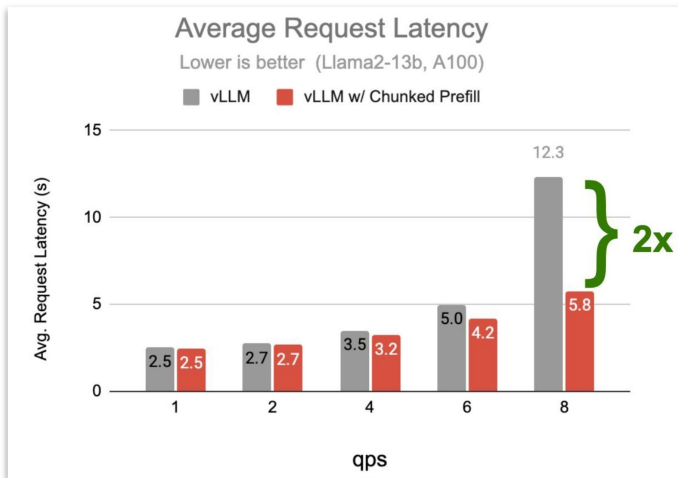
Class hierarchy of vLLM



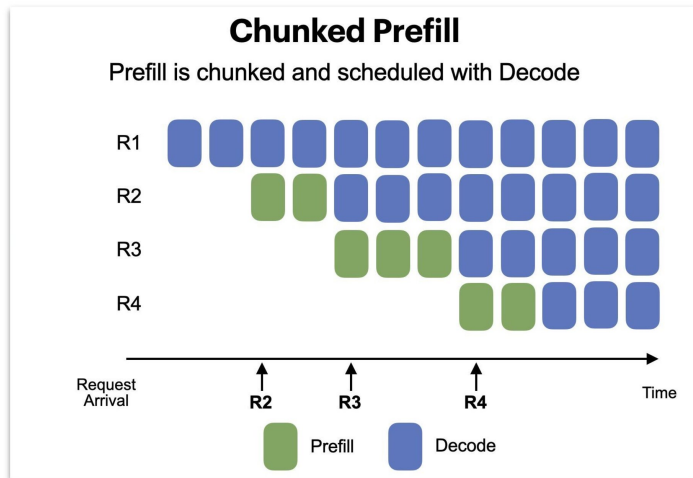
Challenge 1: Prefill vs. Decode asymmetry

Chunk long prefills to protect latency

Chunked Prefill



queries per second



[Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve](#)

Challenge 1: Prefill vs. Decode asymmetry

Prefill-Decode Disaggregation

Next-Level Inference: Why Your Single-Node vLLM Setup Needs Prefill-Decode Disaggregation

April 7, 2026 22 min read

AMD and Embedded LLM

#disaggregation

The Misconception: "Disaggregation is Only for Datacenter Clusters"

TL;DR: Prefill and decode fight over the same GPUs, causing ITL spikes under load. We show how to disaggregate them on a single 8-GPU MI300X node using AMD's MORI-IO connector — achieving **2.5x higher goodput** compared to standard colocated serving on the same 8 GPUs, with stable token generation. Benchmark uses Qwen3-235B-A22B-FP8 at 8 req/s with 2000-token prompts and 1000-token outputs — see Table 3 and [Experimental Details](#) for full configuration.

Challenge 1: Prefill vs. Decode asymmetry

Prefill-Decode Disaggregation

Next-Level Inference: Why Your Single-Node vLLM Setup Needs Prefill-Decode Disaggregation

April 7, 2026 22 min read

AMD and Embedded LLM

#disaggregation

Goodput = maximum request rate (req/s) such that requests satisfy both $TTFT < T_{ttft}$ and $ITL < T_{itl}$.

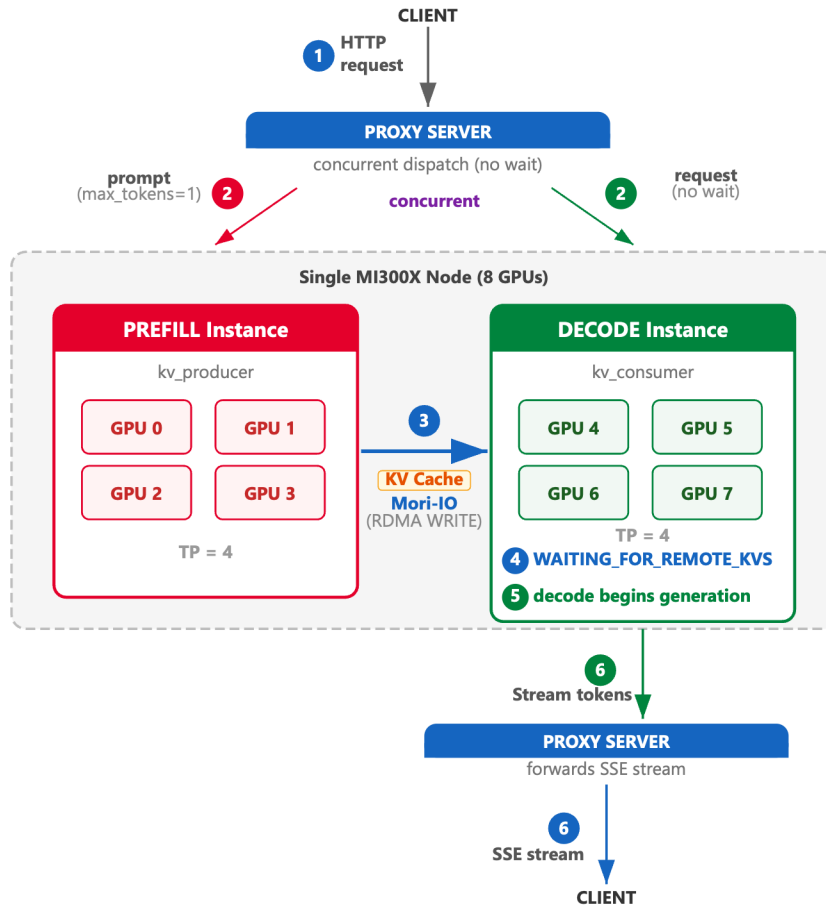
This captures both cost (requests per second) and service quality (latency SLO attainment) in a single number. Our SLO targets: **TTFT < 1 second** and **ITL < 50 ms per token**. A request counts toward **goodput** only if both conditions are met.

Challenge 1: Prefill vs. Decode asymmetry

Prefill-Decode Disaggregation

Write mode request flow. The proxy fires both prefill and decode concurrently (step 2); prefill pushes KV layer-by-layer via RDMA WRITE (step 3) while decode waits.

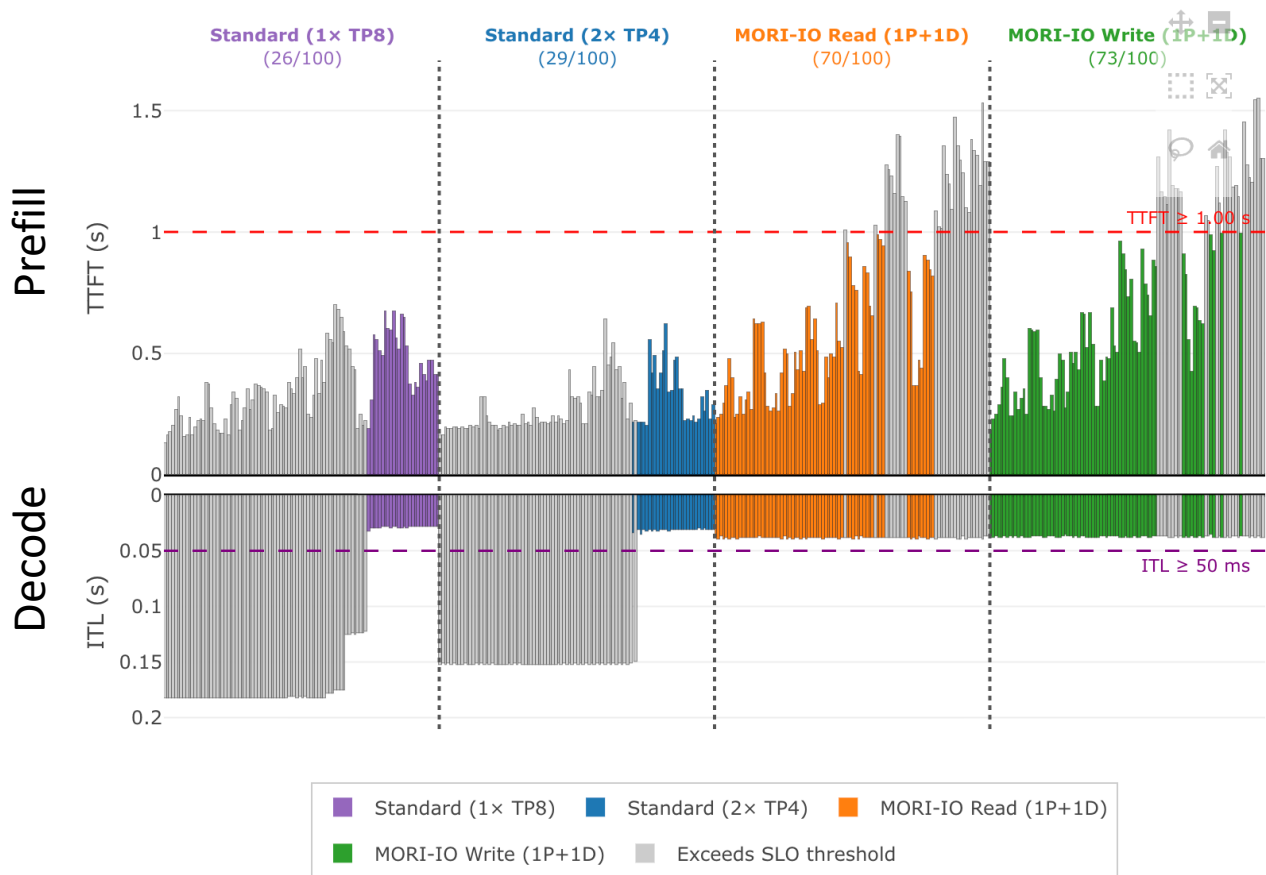
TIME



TTFT threshold:  1.00 s

ITL threshold:  50 ms

TTFT per request (top) / ITL per request (bottom)



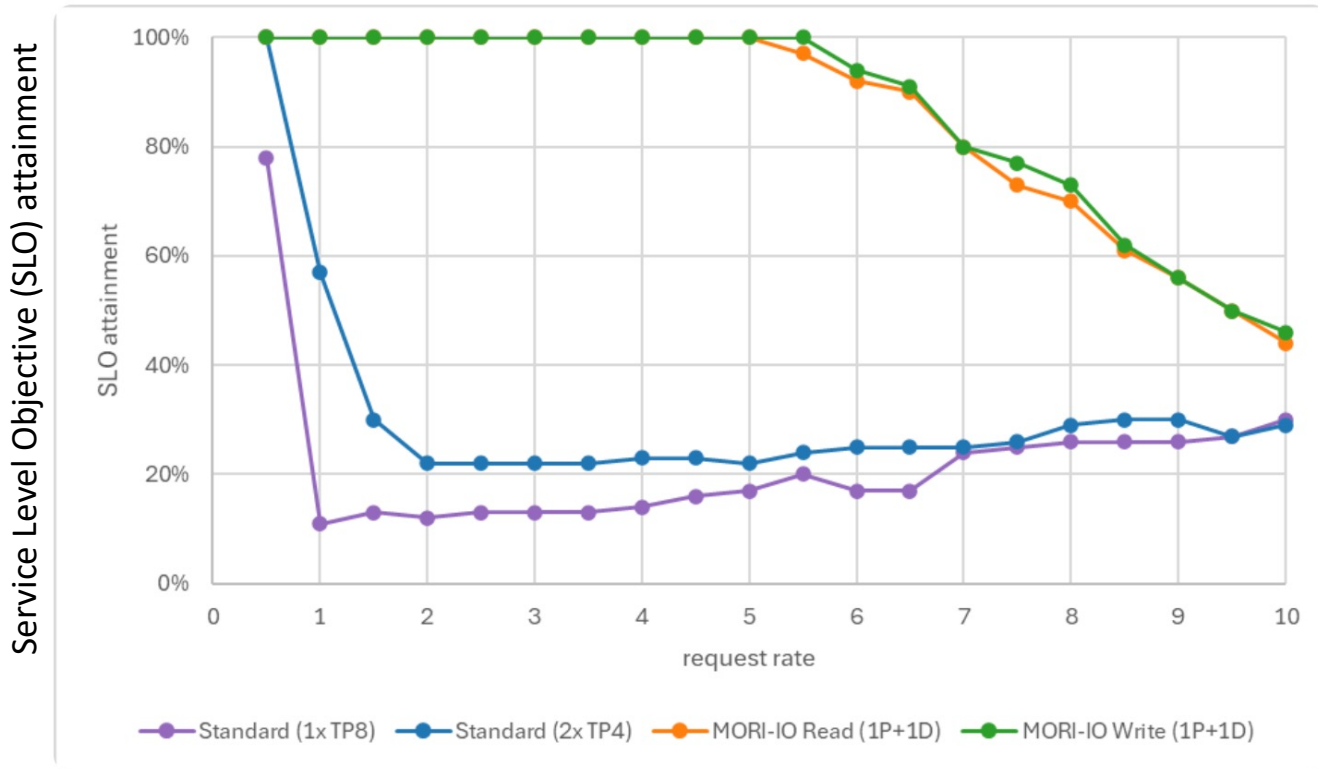


Figure 4: SLO attainment (% of requests meeting both TTFT and ITL targets) across request rates. Both disaggregated modes show higher SLO attainment than all standard serving configurations across all tested request rates.

Challenge 1: Prefill vs. Decode asymmetry

Prefill-Decode Disaggregation

When Should You Use This?

Table 4 summarizes when to prefer each deployment approach.

Your situation	Recommendation
ITL p99 exceeds your SLO under production load	Disaggregate — this is the primary use case
TTFT is your binding constraint (e.g., chatbot UX)	Standard serving may be preferable
High concurrency with long prompts	Disaggregate — prefill interference is worst here
Low request rates with short prompts	Standard serving is sufficient

Challenge 2: KV cache memory waste

KV Cache management with PagedAttention

- **PagedAttention** divides the KV cache into fixed-size blocks.
- Each block contains the attention keys and values for a fixed number of tokens.
- Because blocks do not need to be contiguous in physical memory, vLLM can allocate them dynamically on demand.

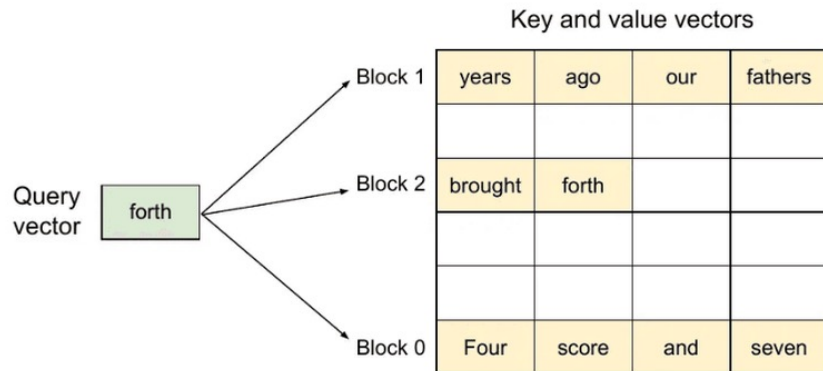


Figure 5. Illustration of the PagedAttention algorithm, where the attention key and values vectors are stored as non-contiguous blocks in the memory.

Challenge 2: KV cache memory waste

Block tables turn logical sequences into physical blocks

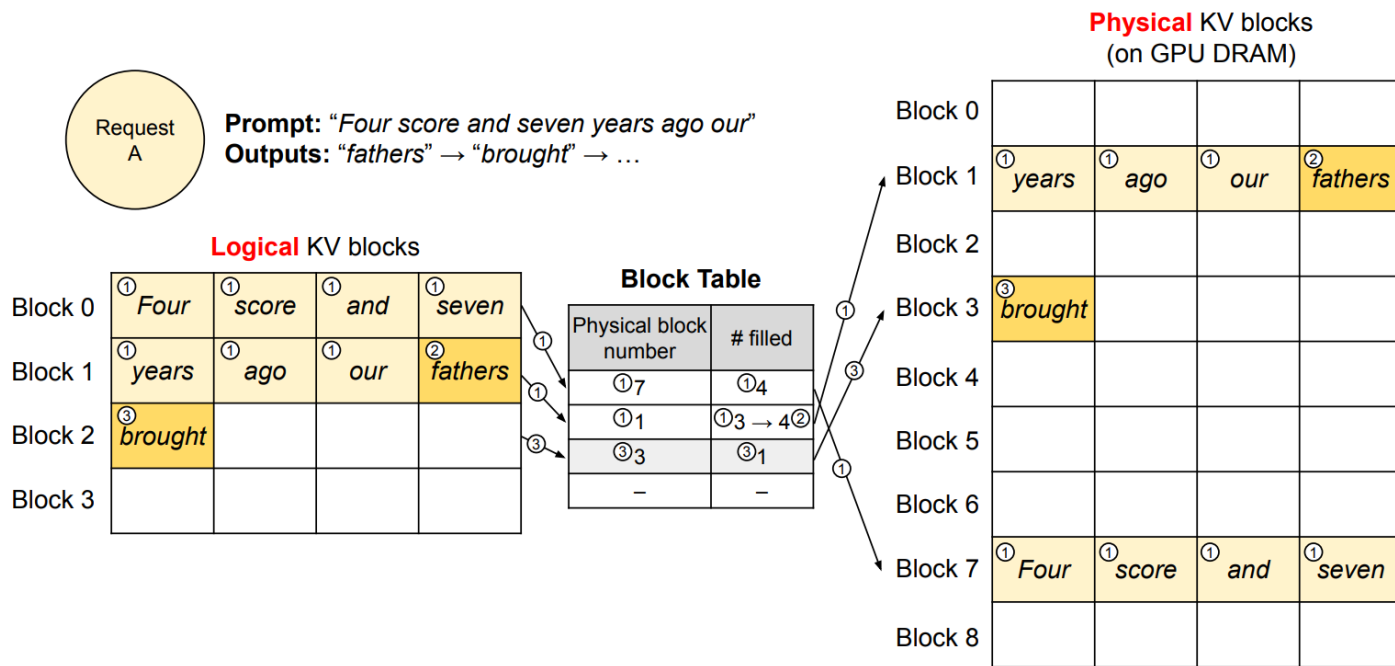


Figure 6. Block table translation in vLLM.

Challenge 2: KV cache memory waste

Allocate only the blocks each request needs

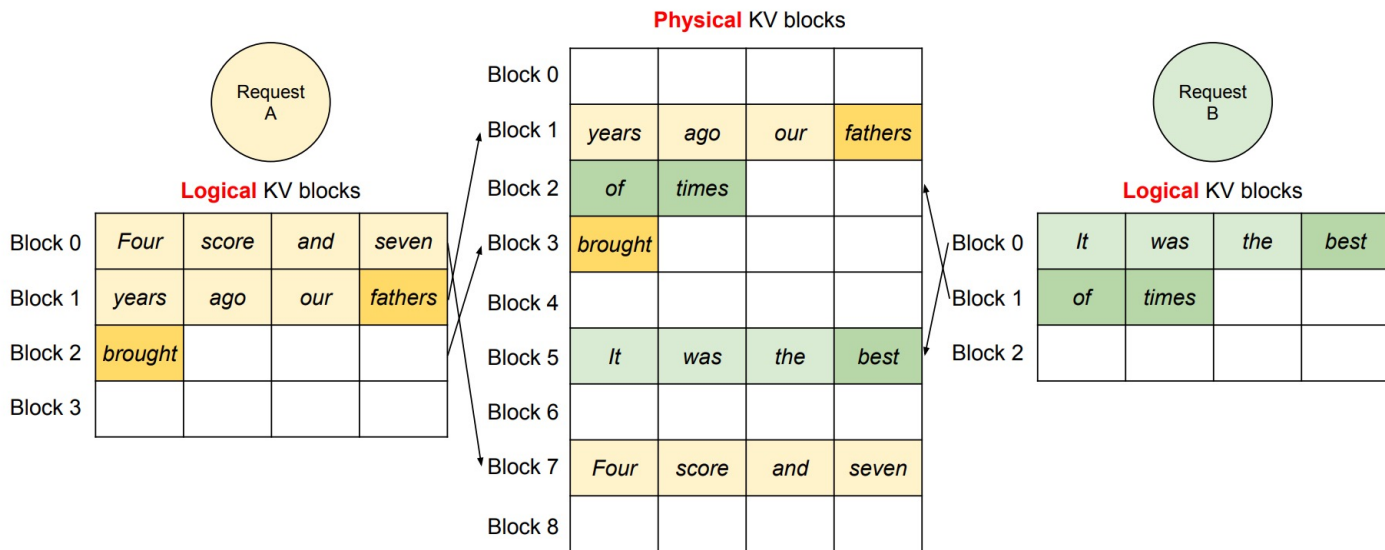
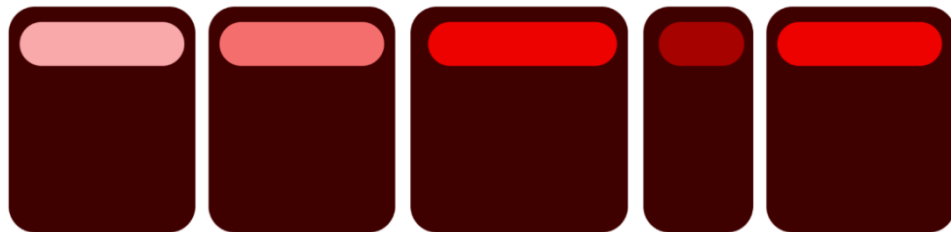


Figure 7. Storing the KV cache of two requests at the same time in vLLM.

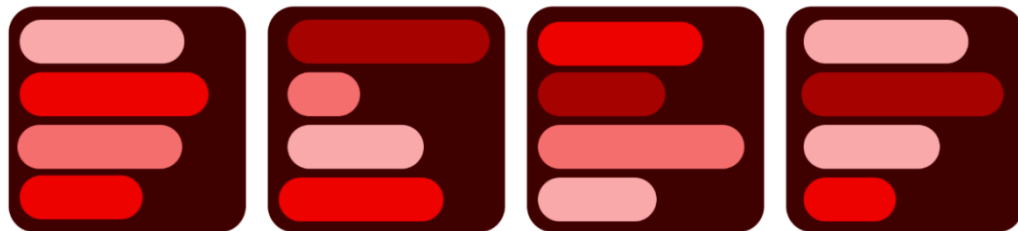
Challenge 3: Batching with varying length of outputs

Continuous batching to increase throughput

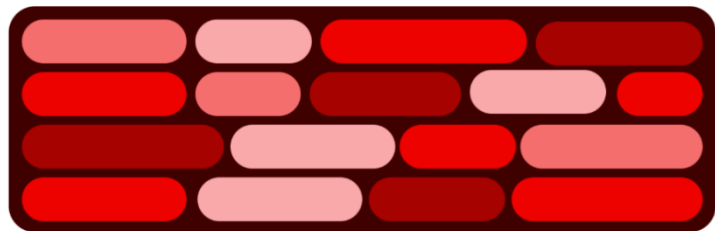
Individual
Requests



Static
Batching

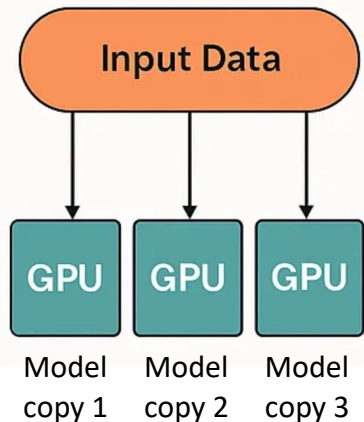


Continuous
Batching

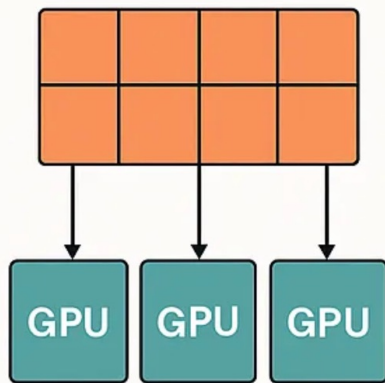


Distributed inference strategies

Data parallelism

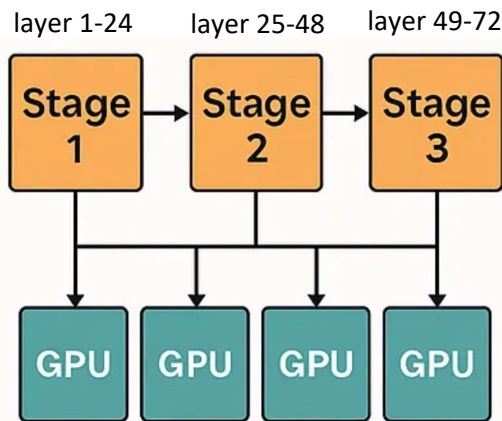


Tensor parallelism

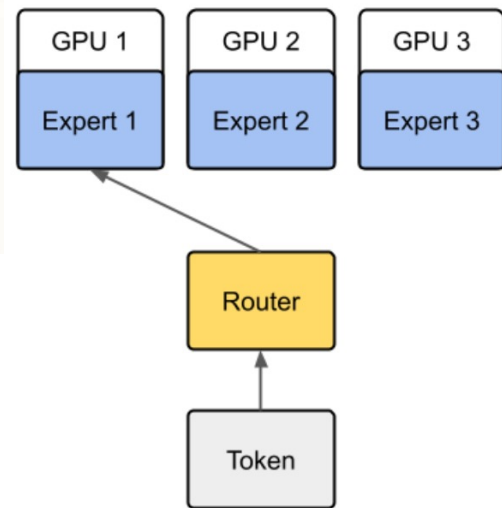


large matrix multiplications
(row-wise/column-wise)

Pipeline parallelism



Expert parallelism



Distributed inference strategies

- **Single GPU (no distributed inference):** if the model fits on a single GPU, distributed inference is probably unnecessary. Run inference on that GPU.
- **Single-node multi-GPU using tensor parallel inference:** if the model is too large for a single GPU but fits on a single node with multiple GPUs, use *tensor parallelism*. For example, set `tensor_parallel_size=4` when using a node with 4 GPUs.
- **Multi-node multi-GPU using tensor parallel and pipeline parallel inference:** if the model is too large for a single node, combine *tensor parallelism* with *pipeline parallelism*. Set `tensor_parallel_size` to the number of GPUs per node and `pipeline_parallel_size` to the number of nodes. For example, set `tensor_parallel_size=8` and `pipeline_parallel_size=2` when using 2 nodes with 8 GPUs per node.

Multi-node deployment

vLLM uses Ray to manage the distributed execution of tasks across multiple nodes and control where execution happens.

The common practice is to set the **tensor parallel size to the number of GPUs in each node, and the pipeline parallel size to the number of nodes.**

if you have 16 GPUs across 2 nodes (8 GPUs per node)

```
vllm serve /path/to/the/model/in/the/container \  
  --tensor-parallel-size 8 \  
  --pipeline-parallel-size 2 \  
  --distributed-executor-backend ray
```



Multi-node deployment

vLLM uses Ray to manage the distributed execution of tasks across multiple nodes and control where execution happens.

The common practice is to set the **tensor parallel size to the number of GPUs in each node, and the pipeline parallel size to the number of nodes.**

if you have 16 GPUs across 2 nodes (8 GPUs per node)

```
vllm serve /path/to/the/model/in/the/container \
  --tensor-parallel-size 8 \
  --pipeline-parallel-size 2 \
  --distributed-executor-backend ray
```

Standard LLM Container Directory

The directory path `/path/to/the/model/in/the/container` typically mirrors a model repository structured like this:

- `config.json` : Model architecture parameters and settings.
- `generation_config.json` : Default decoding parameters (like temperature or top-p)
- `model.safetensors.index.json` : Index mapping model weight shards to specific layers.
- `model-00001-of-00004.safetensors` : First part of the sharded model weights.
- `model-00002-of-00004.safetensors` : Second part of the sharded model weights.
- `model-00003-of-00004.safetensors` : Third part of the sharded model weights.
- `model-00004-of-00004.safetensors` : Fourth part of the sharded model weights.
- `tokenizer_config.json` : Custom configurations for the tokenizer.
- `tokenizer.json` : Main file containing vocabulary token mappings.
- `special_tokens_map.json` : Defines system tokens like `<|endoftext|>` or `<s>`.

Conclusion

- Challenge 1: Prefill vs. Decode asymmetry
 - Chunk long prefills to protect latency
 - Prefill-Decode Disaggregation
- Challenge 2: KV cache memory waste
 - KV Cache management with PagedAttention
- Challenge 3: Batching with varying length of outputs
 - Continuous batching to increase throughput
- Distributed inference strategies